

Spark Data Cleaning, Transformation and Aggregation using DataFrames

Submitted By: Aditya Kumar

Internship: Celebal Technologies – Data Engineering

Objective

To understand Spark fundamentals and perform data cleaning, transformation, and aggregation using Spark DataFrames.

Spark Session was created successfully, and the Ecommerce Sales dataset was loaded into a Spark DataFrame.

```
spark = SparkSession.builder \
    .appName("Celebal Week 5") \
    .getOrCreate()

print("Spark Session Created Successfully!")
```

[2] ✓ 13.3s Python

... Spark Session Created Successfully!

```
df.show(5)
```

Date	Order ID	Customer ID	Product ID	Product Category	Product Name	Quantity Sold	Unit Price	Discount (%)	Payment Method	Customer Location	Order Status	Shipping Cost	Profit Margin	Customer Age	Customer Gender	Customer Segment
2024-01-01	ORD000001	CUST1054	PROD23	Clothing	T-Shirt	5	28.75	15	Cash on Delivery	Los Angeles	Completed	11.12	12.65	59	Non-binary	Basic
2024-01-02	ORD000002	CUST1114	PROD41	Sports	Coffee Maker	1	302.32	5	Debit Card	Chicago	Cancelled	8.37	18.66	44	Male	Basic
2024-01-03	ORD000003	CUST1025	PROD43	Electronics	Lipstick	5	179.39	10	Debit Card	Los Angeles	Returned	17.62	14.95	25	Non-binary	Basic
2024-01-04	ORD000004	CUST1759	PROD18	Home & Kitchen	T-Shirt	3	395.35	15	PayPal	Chicago	Cancelled	6.7	16.66	26	Male	Basic
2024-01-05	ORD000005	CUST1281	PROD45	Sports	Smartphone	1	223.83	0	Credit Card	New York	Returned	19.21	9.64	20	Non-binary	Basic

only showing top 5 rows

```
df.printSchema()
```

```
root
|-- Date: date (nullable = true)
|-- Order ID: string (nullable = true)
|-- Customer ID: string (nullable = true)
|-- Product ID: string (nullable = true)
|-- Product Category: string (nullable = true)
|-- Product Name: string (nullable = true)
|-- Quantity Sold: integer (nullable = true)
|-- Unit Price: double (nullable = true)
|-- Discount (%): integer (nullable = true)
|-- Payment Method: string (nullable = true)
|-- Customer Location: string (nullable = true)
|-- Order Status: string (nullable = true)
|-- Shipping Cost: double (nullable = true)
|-- Profit Margin: double (nullable = true)
|-- Customer Age: integer (nullable = true)
|-- Customer Gender: string (nullable = true)
|-- Customer Segment: string (nullable = true)
|-- Review Rating: integer (nullable = true)
|-- Total Sales: double (nullable = true)
|-- Discounted Price: double (nullable = true)
```

```
print("Total Records:", df.count())
print("Total Columns:", len(df.columns))
```

[6] ✓ 0.7s

... Total Records: 100
Total Columns: 20

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer:

Traditional MapReduce processes data by writing intermediate results to disk after every stage, which makes it slower for iterative and real-time applications. It also requires multiple jobs for complex workflows, increasing execution time. Apache Spark overcomes these limitations by using in-memory processing, reducing disk I/O and providing much faster execution. Spark also supports libraries for SQL, Machine Learning, Graph Processing, and Streaming within a single framework, making it more efficient and easier to use for modern big data applications.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer:

Spark stores intermediate data in memory (RAM) instead of writing it to disk after every operation. Since iterative machine learning algorithms repeatedly access the same dataset, keeping the data in memory significantly reduces disk read/write operations. This results in faster execution, lower latency, and improved performance compared to traditional disk-based systems like MapReduce.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

Answer:

The dataset contains the columns **Customer ID** and **Date**, which are used as equivalents of **user_id** and **transaction_date**.

PySpark Code:

```
duplicate_removed_df = df.dropDuplicates(["Customer ID", "Date"])

print("Original Records :", df.count())
print("Records After Removing Duplicates :", duplicate_removed_df.count())

duplicate_removed_df.show(5)
```

✓ 25s Python

Original Records : 100
Records After Removing Duplicates : 100

Date	Order ID	Customer ID	Product ID	Product Category	Product Name	Quantity Sold	Unit Price	Discount (%)	Payment Method	Customer Location	Order Status	Shipping Cost	Profit Margin	Customer Age	Customer Gender	Customer
2024-02-08	ORD000039	CUST1006	PROD42	Clothing	Coffee Maker	2	112.99	15	Debit Card	Los Angeles	Cancelled	19.71	20.51	59	Female	
2024-01-03	ORD000003	CUST1025	PROD43	Electronics	Lipstick	5	179.39	10	Debit Card	Los Angeles	Returned	17.62	14.95	25	Non-binary	
2024-01-25	ORD000025	CUST1027	PROD50	Electronics	Coffee Maker	3	258.49	15	Credit Card	Chicago	Returned	9.01	12.18	49	Male	
2024-01-19	ORD000019	CUST1030	PROD41	Home & Kitchen	Lipstick	5	474.95	0	Credit Card	Houston	Cancelled	7.05	14.14	44	Non-binary	
2024-01-18	ORD000018	CUST1032	PROD18	Electronics	T-Shirt	3	221.34	5	Credit Card	Chicago	Returned	16.19	18.81	52	Female	

only showing top 5 rows

Explanation:

The `dropDuplicates()` function removes duplicate records based on the combination of Customer ID and Date while keeping the first occurrence of each unique record.

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Answer:

Since the dataset stores city names instead of regions, **Los Angeles** is used to represent a location for filtering. Also, **Product Category** is used as **product_category**, and **Total Sales** as **sale_amount**.

PySpark Code

```
from pyspark.sql.functions import avg

filtered_sales = (
    df.filter(col("Customer Location") == "Los Angeles")
      .groupBy("Product Category")
      .agg(avg("Total Sales").alias("Average Sales"))
)

filtered_sales.show()
```

✓ 0.7s Python

Product Category	Average Sales
Home & Kitchen	929.61
Sports	1595.72
Electronics	992.9599999999999
Clothing	308.0028571428571
Beauty	1135.48

Explanation

The DataFrame is first filtered to include only records from the selected location. The filtered data is then grouped by Product Category, and the average value of Total Sales is calculated for each category.

Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string "Unknown".

Answer:

- .na.drop() removes rows that contain null values.
- .na.fill() replaces null values with a specified value without deleting the rows.

In this dataset, the **Order Status** column is used as the status column.

PySpark Code

```
# Create a sample null value for demonstration
demo_df = df.withColumn(
    "Order Status",
    when(col("Order ID") == "ORD00005", None)
    .otherwise(col("Order Status"))
)

filled_df = demo_df.na.fill({"Order Status": "Unknown"})

filled_df.select("Order ID", "Order Status").show(10)
```

✓ 0.2s Python

Order ID	Order Status
ORD000001	Completed
ORD000002	Cancelled
ORD000003	Returned
ORD000004	Cancelled
ORD000005	Unknown
ORD000006	Cancelled
ORD000007	Cancelled
ORD000008	Returned
ORD000009	Returned
ORD000010	Cancelled

only showing top 10 rows

Explanation

First, a sample null value is introduced for demonstration purposes. Then, .na.fill() replaces all null values in the Order Status column with **"Unknown"**.

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Answer:

The provided dataset contains only **100 records**. Therefore, for demonstration purposes, the condition has been modified from **count > 100** to **count > 20**.

PySpark Code

```
city_count = (
    df.groupBy("Customer Location")
      .count()
      .filter(col("count") > 20)
)

city_count.show()
```

✓ 0.5s Python

Customer Location	count
Los Angeles	22
New York	22

Explanation

The data is grouped by Customer Location, and the total number of records is calculated using the count() function. Since the dataset contains only 100 records, the threshold was adjusted to display meaningful output.

Q7: How does the immutability of Spark DataFrames affect how you perform data cleaning steps like dropping columns or renaming them?

Answer:

Spark DataFrames are immutable, which means they cannot be modified after they are created. Any data cleaning operation such as dropping columns, renaming columns, or filtering rows does not change the original DataFrame. Instead, Spark creates and returns a new DataFrame containing the applied changes. This approach ensures data consistency, supports fault tolerance, and enables optimization through Spark's execution engine.

PySpark Code

```
new_df = df.drop("Discount (%)")
renamed_df = df.withColumnRenamed("Customer Age", "Age")

print("Original Columns:")
print(df.columns)

print("\nRenamed DataFrame Columns:")
print(renamed_df.columns)
```

✓ 0.0s Python

Original Columns:
[Date, Order ID, Customer ID, Product ID, Product Category, Product Name, Quantity Sold, Unit Price, Discount (%), Payment Method, Customer Location, Order Status, Shipping Cost, Profit Margin, ...]

Renamed DataFrame Columns:
[Date, Order ID, Customer ID, Product ID, Product Category, Product Name, Quantity Sold, Unit Price, Discount (%), Payment Method, Customer Location, Order Status, Shipping Cost, Profit Margin, ...]

Explanation

In the above example, the original DataFrame remains unchanged. The variables new_df and renamed_df store the modified DataFrames.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is "Premium".

Answer:

The dataset contains the **Customer Age** column but does not contain a **Subscription** column. Therefore, a temporary Subscription column is created for demonstration purposes.

PySpark Code

```
subscription_df = df.withColumn(
    "Subscription",
    when(col("Customer Age").between(18, 30), "Premium")
    .otherwise("Standard")
)

filtered_df = subscription_df.filter(
    (col("Customer Age").between(18, 30)) &
    (col("Subscription") == "Premium")
)

filtered_df.select(
    "Customer ID",
    "Customer Age",
    "Subscription",
    "Product Category",
    "Total Sales"
).show()
```

Result

Customer ID	Customer Age	Subscription	Product Category	Total Sales
CUST1025	25	Premium	Electronics	896.95
CUST1759	26	Premium	Home & Kitchen	1186.05
CUST1281	20	Premium	Sports	223.83
CUST1250	20	Premium	Sports	984.58
CUST1794	25	Premium	Home & Kitchen	103.14
CUST1104	24	Premium	Clothing	158.75
CUST1913	26	Premium	Electronics	1215.57
CUST1223	27	Premium	Clothing	946.05
CUST1616	24	Premium	Sports	1804.08
CUST1651	20	Premium	Sports	563.32
CUST1284	24	Premium	Sports	208.98
CUST1825	21	Premium	Clothing	307.26
CUST1432	30	Premium	Home & Kitchen	1705.3
CUST1348	25	Premium	Beauty	520.96
CUST1159	23	Premium	Beauty	960.64
CUST1094	19	Premium	Electronics	111.52
CUST1389	21	Premium	Sports	952.96
CUST1867	26	Premium	Sports	2026.1
CUST1270	22	Premium	Electronics	984.26
CUST1996	27	Premium	Sports	594.45

Explanation

A temporary Subscription column is created for demonstration. The data is then filtered to include only customers aged **18–30** having a Premium subscription.

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Answer:

Handling null values before performing aggregation is important because null values can affect the accuracy and reliability of the results. Replacing or removing null values ensures that functions such as sum(), avg(), min(), and max() produce meaningful and consistent outputs. It also prevents unexpected results during data analysis.

PySpark Code

```
# Create a sample null value for demonstration
sales_df = df.withColumn(
    "Total Sales",
    when(col("Order ID") == "ORD000008", None)
    .otherwise(col("Total Sales"))
)

clean_sales_df = sales_df.na.fill({"Total Sales": 0})

clean_sales_df.groupby("Product Category") \
    .sum("Total Sales") \
    .show()
```

Explanation

In the above example, null values in the Total Sales column are replaced with **0** before calculating the sum for each Product Category.

Q10: Write the code to revise a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

Answer:

The dataset does not contain a `raw_timestamp` column. Therefore, the **Date** column is used as an equivalent timestamp field.

PySpark Code

```
from pyspark.sql.functions import to_timestamp

timestamp_df = df.withColumn(
    "event_time",
    to_timestamp("Date", "yyyy-MM-dd")
).drop("Date")

timestamp_df.select("event_time").show(10)

timestamp_df.printSchema()
```

✓ 0.2s

```
-----+-----
|          event_time          |
+-----+-----
|2024-01-01 00:00:00|
|2024-01-02 00:00:00|
|2024-01-03 00:00:00|
|2024-01-04 00:00:00|
|2024-01-05 00:00:00|
|2024-01-06 00:00:00|
|2024-01-07 00:00:00|
|2024-01-08 00:00:00|
|2024-01-09 00:00:00|
|2024-01-10 00:00:00|
+-----+-----
only showing top 10 rows
root
 |-- Order ID: string (nullable = true)
 |-- Customer ID: string (nullable = true)
 |-- Product ID: string (nullable = true)
 |-- Product Category: string (nullable = true)
 |-- Product Name: string (nullable = true)
 |-- Quantity Sold: integer (nullable = true)
 |-- Unit Price: double (nullable = true)
 |-- Discount (%): integer (nullable = true)
 |-- Payment Method: string (nullable = true)
 ...
 |-- Total Sales: double (nullable = true)
 |-- Discounted Price: double (nullable = true)
 |-- event_time: timestamp (nullable = true)
```

Explanation

The `Date` column is converted into `TimestampType` using `to_timestamp()`. A new column named `event_time` is created, and the original `Date` column is removed.

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Answer:

A shuffle is the process of redistributing data across different partitions in a Spark cluster. It occurs during operations such as `groupBy()`, `join()`, and `reduceByKey()`, where related records must be brought together before processing.

A wide transformation requires data to move between partitions, unlike a narrow transformation, where each partition can be processed independently. Since shuffling involves network communication and disk I/O, it is one of the most expensive operations in Spark and can affect application performance.

PySpark Code

```
df.groupBy("Product Category") \
  .sum("Total Sales") \
  .show()
```

✓ 0.3s

```
-----+-----+-----
|Product Category| sum(Total Sales)|
+-----+-----+-----
| Home & Kitchen|13858.380000000001|
|   Sports      |19059.489999999998|
| Electronics   |   15271.19       |
| Clothing      |14132.649999999998|
| Beauty        |14609.570000000002|
+-----+-----+-----
```

Explanation

In the above operation, Spark performs a shuffle to bring records of the same `Product Category` together before calculating the total sales.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

Answer:

The dataset does not contain **Email** and **Username** columns. Therefore, these columns are created temporarily for demonstration.

PySpark Code

```
user_df = df.withColumn(
    "Email",
    lit("customer@example.com")
).withColumn(
    "Username",
    lit("customer")
)

user_df = user_df.withColumn(
    "Email",
    when(col("Customer ID") == "CUST001", None)
    .otherwise(col("Email"))
)

user_df = user_df.withColumn(
    "Username",
    when(col("Customer ID") == "CUST002", "")
    .otherwise(col("Username"))
)

clean_user_df = user_df.filter(
    col("Email").isNotNull() &
    (col("Username") != "")
)

clean_user_df.select(
    "Customer ID",
    "Email",
    "Username"
).show()
```

Result

```
+-----+-----+-----+
|Customer ID|      Email|Username|
+-----+-----+-----+
|CUST1654|customer@example.com|customer|
|CUST1114|customer@example.com|customer|
|CUST1025|customer@example.com|customer|
|CUST1759|customer@example.com|customer|
|CUST1281|customer@example.com|customer|
|CUST1250|customer@example.com|customer|
|CUST1228|customer@example.com|customer|
|CUST1142|customer@example.com|customer|
|CUST1754|customer@example.com|customer|
|CUST1104|customer@example.com|customer|
|CUST1692|customer@example.com|customer|
|CUST1758|customer@example.com|customer|
|CUST1913|customer@example.com|customer|
|CUST1558|customer@example.com|customer|
|CUST1089|customer@example.com|customer|
|CUST1604|customer@example.com|customer|
|CUST1432|customer@example.com|customer|
|CUST1032|customer@example.com|customer|
|CUST1030|customer@example.com|customer|
|CUST1095|customer@example.com|customer|
+-----+-----+-----+
only showing top 20 rows
```

Explanation

Temporary Email and Username columns are created. Rows containing null emails or empty usernames are removed using filter conditions.

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Answer:

The .agg() function allows multiple aggregate operations to be performed in a single query.

PySpark Code

```
df.agg(
    min("Unit Price").alias("Minimum Price"),
    max("Unit Price").alias("Maximum Price"),
    avg("Unit Price").alias("Average Price")
).show()
✓ 0.2s
```

Minimum Price	Maximum Price	Average Price
28.75	492.28	283.79790000000014

Explanation

The above query calculates the minimum, maximum, and average values of the Unit Price column simultaneously.

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Answer:

When inferSchema=true is used, Spark automatically detects the data type of each column. If the dataset contains inconsistent date formats, Spark may infer the wrong data type or interpret some values as null. This can lead to incorrect analysis, failed transformations, and data quality issues. Therefore, it is often better to define the schema explicitly when working with inconsistent datasets.

PySpark Code

```
df.printSchema()
✓ 0.0s
```

```
root
 |-- Date: date (nullable = true)
 |-- Order ID: string (nullable = true)
 |-- Customer ID: string (nullable = true)
 |-- Product ID: string (nullable = true)
 |-- Product Category: string (nullable = true)
 |-- Product Name: string (nullable = true)
 |-- Quantity Sold: integer (nullable = true)
 |-- Unit Price: double (nullable = true)
 |-- Discount (%): integer (nullable = true)
 |-- Payment Method: string (nullable = true)
 |-- Customer Location: string (nullable = true)
 |-- Order Status: string (nullable = true)
 |-- Shipping Cost: double (nullable = true)
 |-- Profit Margin: double (nullable = true)
 |-- Customer Age: integer (nullable = true)
 |-- Customer Gender: string (nullable = true)
 |-- Customer Segment: string (nullable = true)
 |-- Review Rating: integer (nullable = true)
 |-- Total Sales: double (nullable = true)
 |-- Discounted Price: double (nullable = true)
```

Explanation

Always verify the inferred schema before performing transformations.

Q15: Write a final processing pipeline that filters out duplicates, fills null prices with 0, groups by store_id to calculate total revenue.

Answer:

The dataset does not contain a **store_id** column. Therefore, **Product Category** is used for grouping to demonstrate the processing pipeline.

PySpark Code

```
pipeline_df = df.dropDuplicates()

pipeline_df = pipeline_df.na.fill({
    "Unit Price": 0
})

result_df = pipeline_df.groupBy("Product Category") \
    .agg(
        sum("Total Sales").alias("Total Revenue")
    )

result_df.show()
✓ 1.0s
```

Product Category	Total Revenue
Home & Kitchen	13858.38
Sports	19059.489999999998
Electronics	15271.19
Clothing	14132.65
Beauty	14609.570000000002

Conclusion

In this assignment, Apache Spark DataFrames were used to perform various data processing operations, including data cleaning, filtering, aggregation, schema modification, and grouping. The assignment demonstrated the advantages of Spark's in-memory processing and immutable DataFrame architecture while building a complete data processing pipeline using PySpark.
